1 MDS Extensibility Framework

1.1 Overview

You use the Extensibility Framework to create extensions that you have programmed using the .NET Framework. You can also use the predefined extensions that are provided in the standard.

Via extensions, you can add functions to the MOC that are not available via simple configuration. You can call the extensions in the MOC toolbar if configured accordingly. You can integrate separate applications into the MOC menu.

The programming environment for customizations is the .NET Framework of Microsoft. You can use any programming language that is supported by .NET Framework. The programming language used in the examples of this document is C#.

Examples

To customize the MOC, the following options are available:

1. Dynamic behavior of application
   - Specific logic to visualize/enable command buttons in the toolbar
   - Specific logic to validate input fields before requesting data
   - Specific logic with/after entry of values into an input field
2. Extension of the toolbar with external self-programmed functions
3. Development of entirely own applications (free programming) that can be integrated into the MOC (menu and transaction code). Internal functions of the MOC are provided, e.g. web services, error logging, system information, reporting, translation libraries, etc. You can use all possibilities of the .NET Framework and you can also use third-party components.

1.2 Dependency Injection

If you develop own customizations, you require objects that provide basic functions (logging, calling web service, printing report, etc.).

These objects are passed in the constructors of the classes. Contrary to the *classic* object-oriented programming, you need not define the objects in a fixed order and number. The objects that are passed to the constructor are dynamically "injected".

For further information on the *Dependency injection*, refer to the relevant literature.

Examples:

```
// Constructor without parameter.
public constructorExample()
{
}


// Constructor with parameter, IRequestFactory is injected at run time.
public constructorExample1 (IRequestFactory requestFactory)
{
    this.requestFactory = requestFactory;
}
```

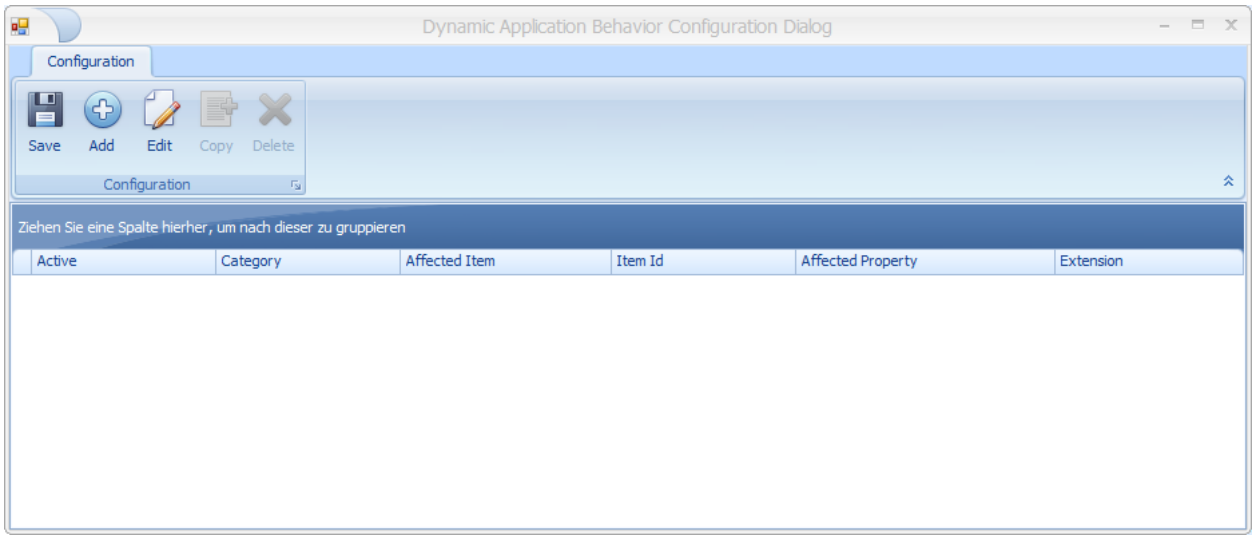
1.3 Extended application configuration

1.3.1 Configuration

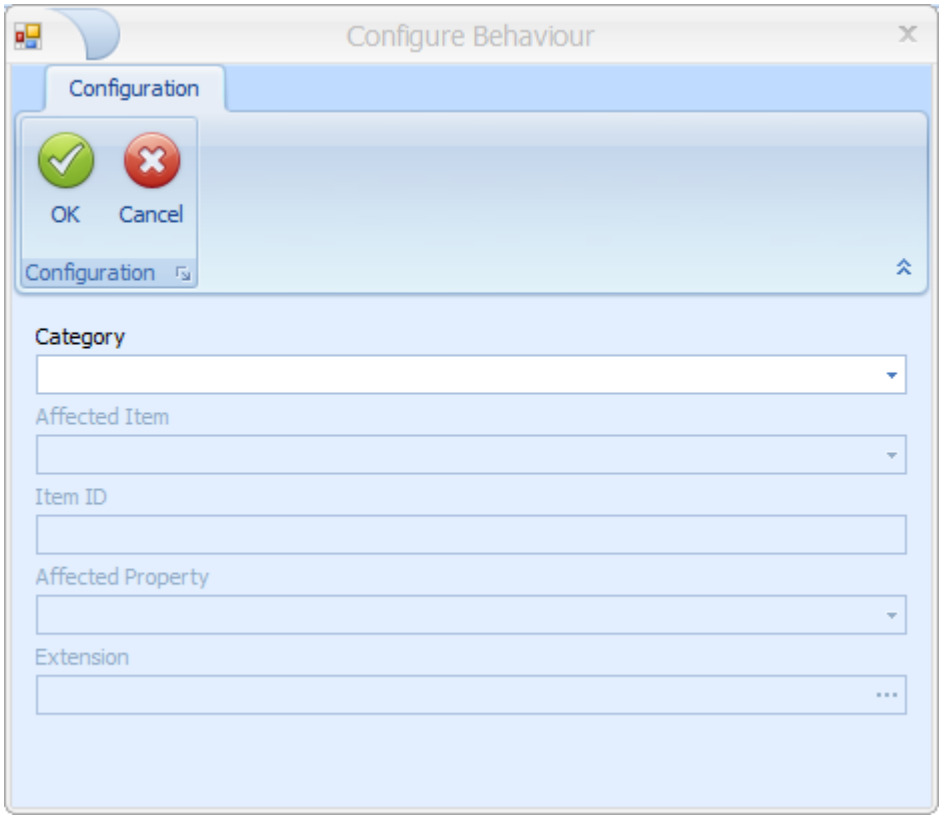
You can configure an extension using the button *Configure dynamic behavior of application* in the toolbar of an application. This function is only available if the MES Development Suite is available and enabled.

The configured extensions made to an application are saved in the file ***DynamicApplicationBehaviors.config*** in the directory of the respective application.

Example: <MOC directory>\local\conf\MOC\Apps\OrderOverview\DynamicApplicationBehaviors.config

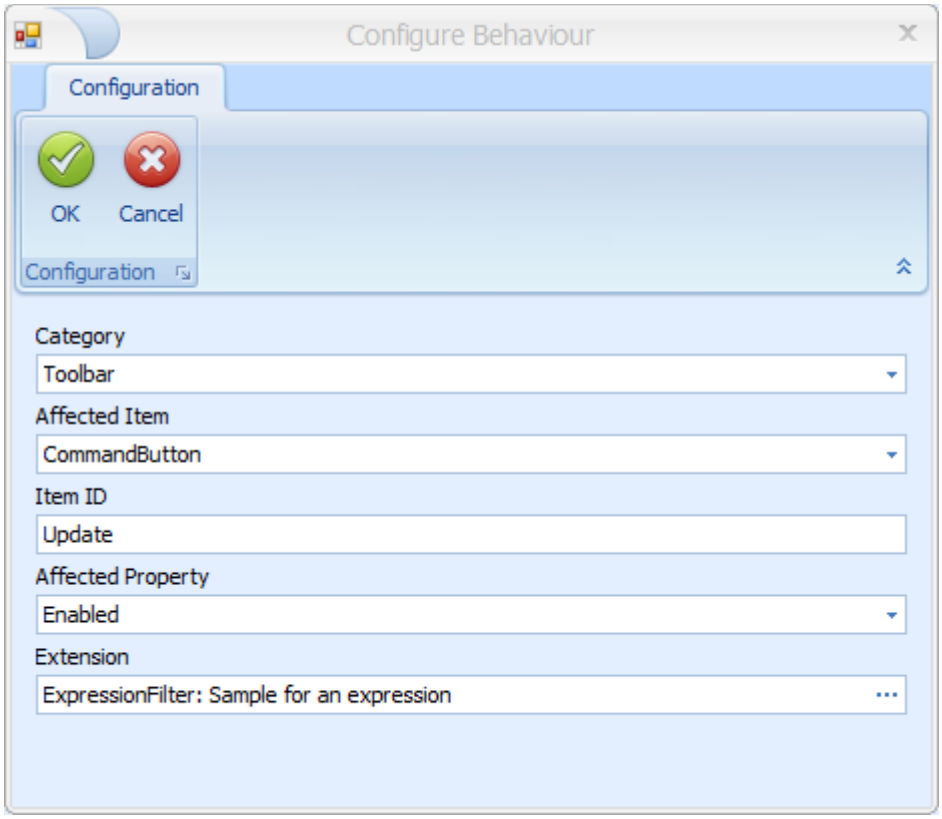


Add/Change a configuration



- **Category**:
  - Toolbar
  - SelectionPanel (input fields selection criteria)
- **AffectedItem** (depending on Category):
  - Toolbar:
    - CommandButton
  - SelectionPanel:
    - Validation (check when data is requested)
    - EditControl (check when data is entered)
- **Item ID**: ID of the affected item (depending on AffectedItem)
  - CommandButton: ID of the function in the link editor (e.g. OrderInformation in the Order overview)
  - EditControl: Fieldname of the input field (e.g. order.id in the Order overview)
- **Affected Property** (only with **AffectedItem** = CommandButton)
  - Visible
  - Enabled

- **Extension:** Selection of the extension in a further dialog



1.3.2 Available standard extensions

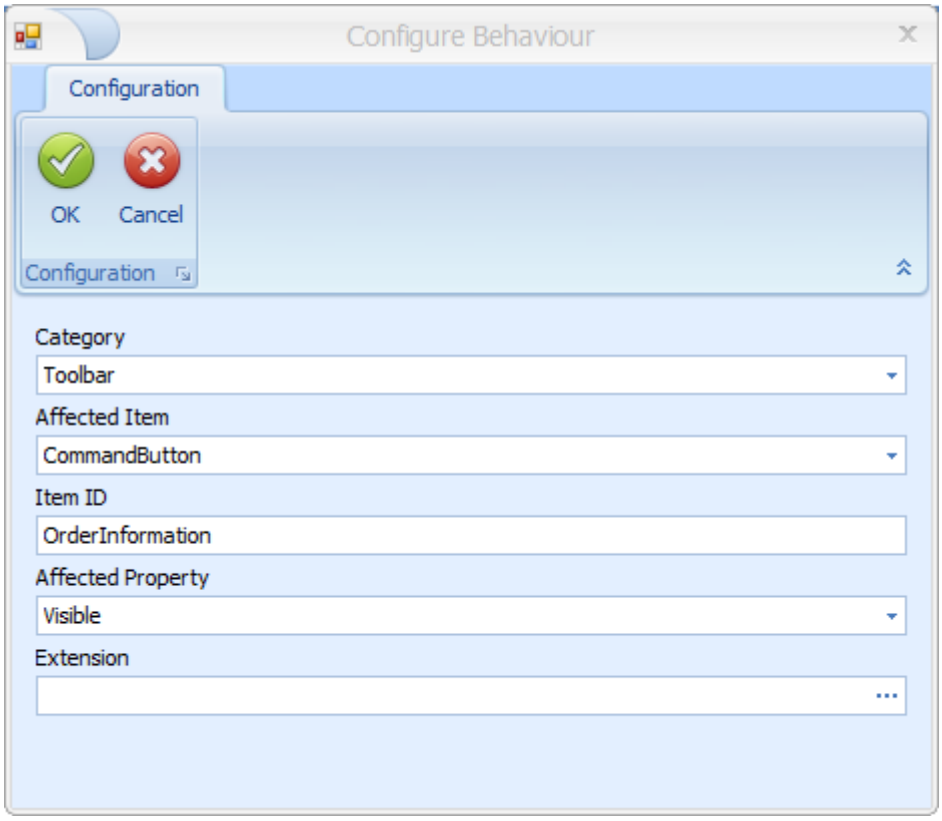
1.3.2.1 Visible/Enabled for toolbar functions

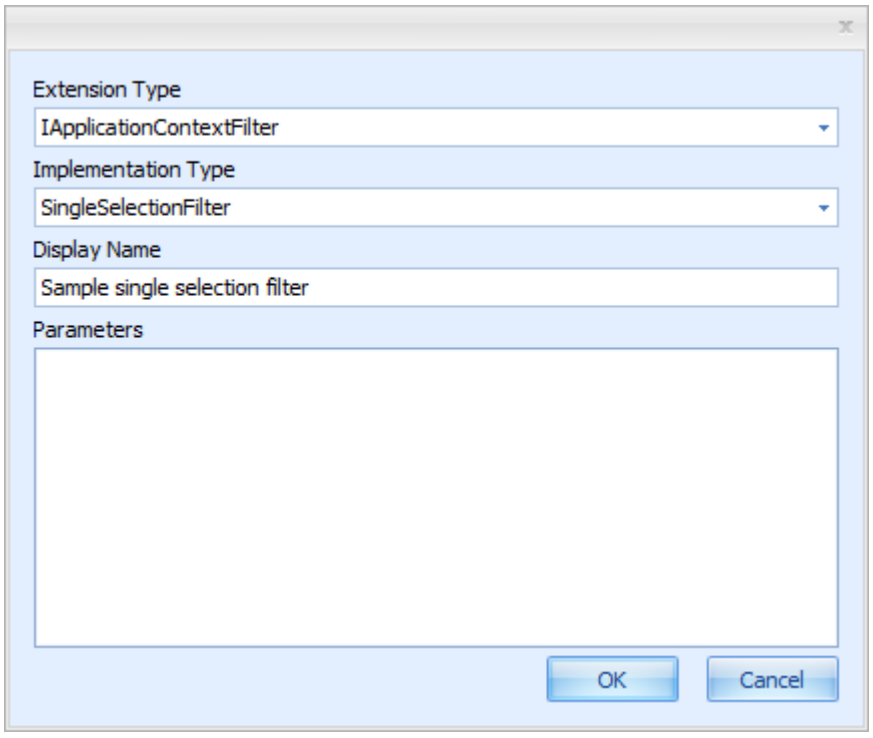
- **Category**: Toolbar
- **AffectedItem:** CommandButton
- **Item ID**: ID of the function (e.g. "OrderInformation" in the Order overview)
- **Affected Property**: Visible or Enabled
  - Visible
  - Enabled
- **Extension:**
  - ExtensionType: IApplicationContextFilter
  - Implementation Type:
    - SingleSelectionType
    - ActivePluginFilter
    - SelectedRowCountFilter
    - AnySelectionFilter
    - MultiSelectionFilter
    - InverseAuthorisation

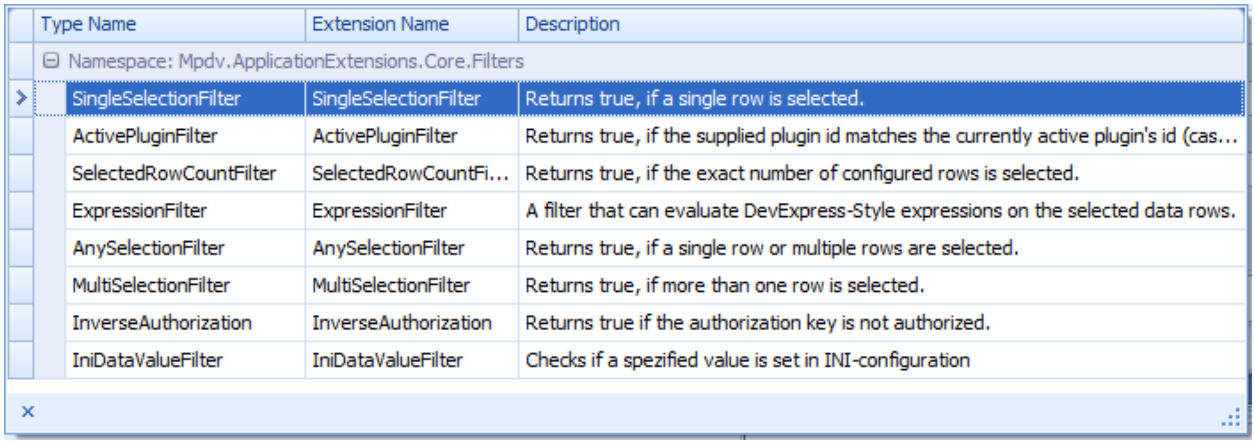
- IniDataValueFilter

Example 1:

The function *Order information* in the *Order overview* is not visible if more than 1 order is selected.



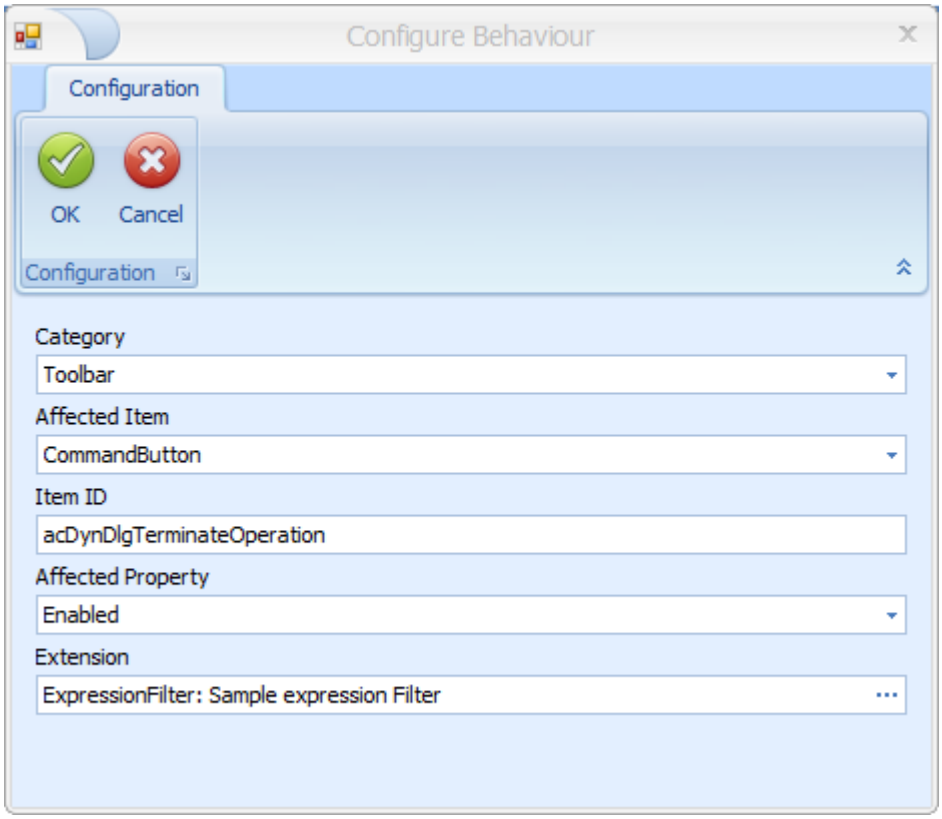


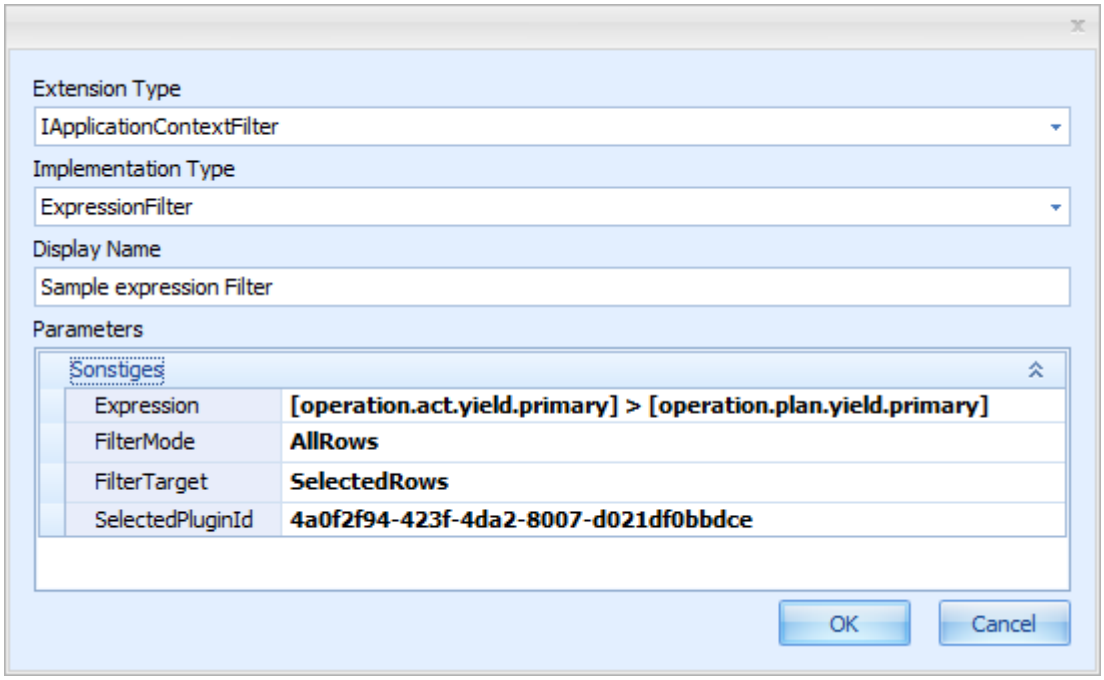


Type Name	Extension Name	Description
Namespace: Mpdv.ApplicationExtensions.Core.Filters		
SingleSelectionFilter	SingleSelectionFilter	Returns true, if a single row is selected.
ActivePluginFilter	ActivePluginFilter	Returns true, if the supplied plugin id matches the currently active plugin's id (cas...
SelectedRowCountFilter	SelectedRowCountFi...	Returns true, if the exact number of configured rows is selected.
ExpressionFilter	ExpressionFilter	A filter that can evaluate DevExpress-Style expressions on the selected data rows.
AnySelectionFilter	AnySelectionFilter	Returns true, if a single row or multiple rows are selected.
MultiSelectionFilter	MultiSelectionFilter	Returns true, if more than one row is selected.
InverseAuthorization	InverseAuthorization	Returns true if the authorization key is not authorized.
IniDataValueFilter	IniDataValueFilter	Checks if a spezified value is set in INI-configuration

Example 2:

The function *Terminate operation* in the *Order overview* is only enabled if the yield posted (P) is greater than the target quantity (P). You use the filter options of the table view (grid) to this end (ExpressionFilter). Extension: ExpressionFilter.



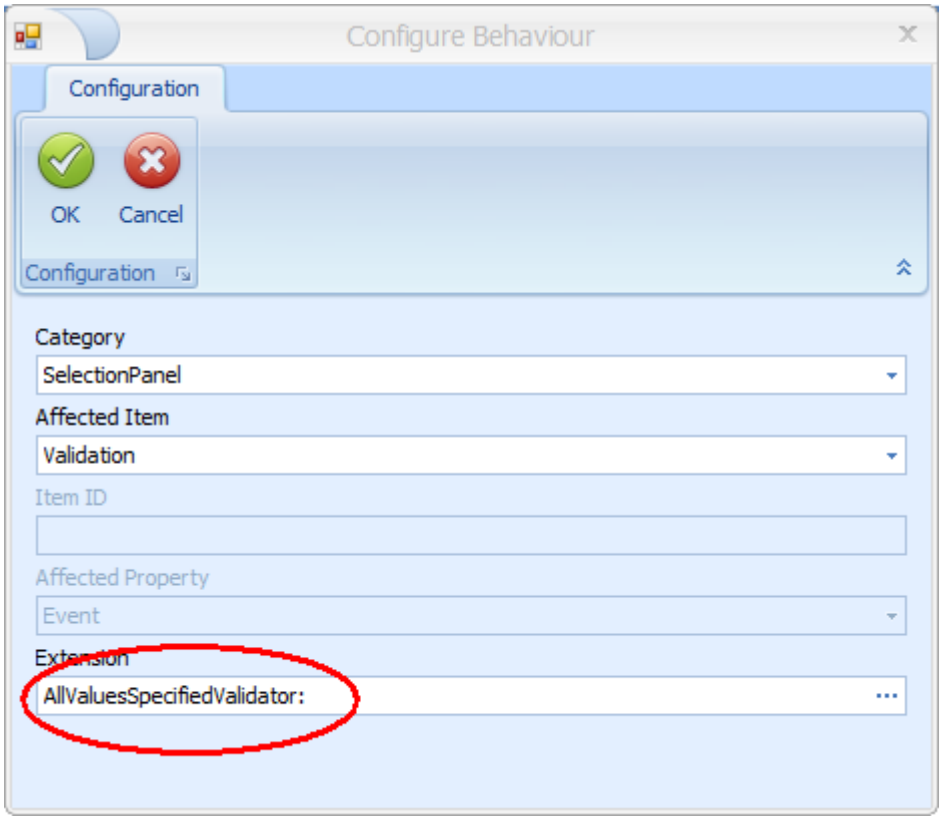


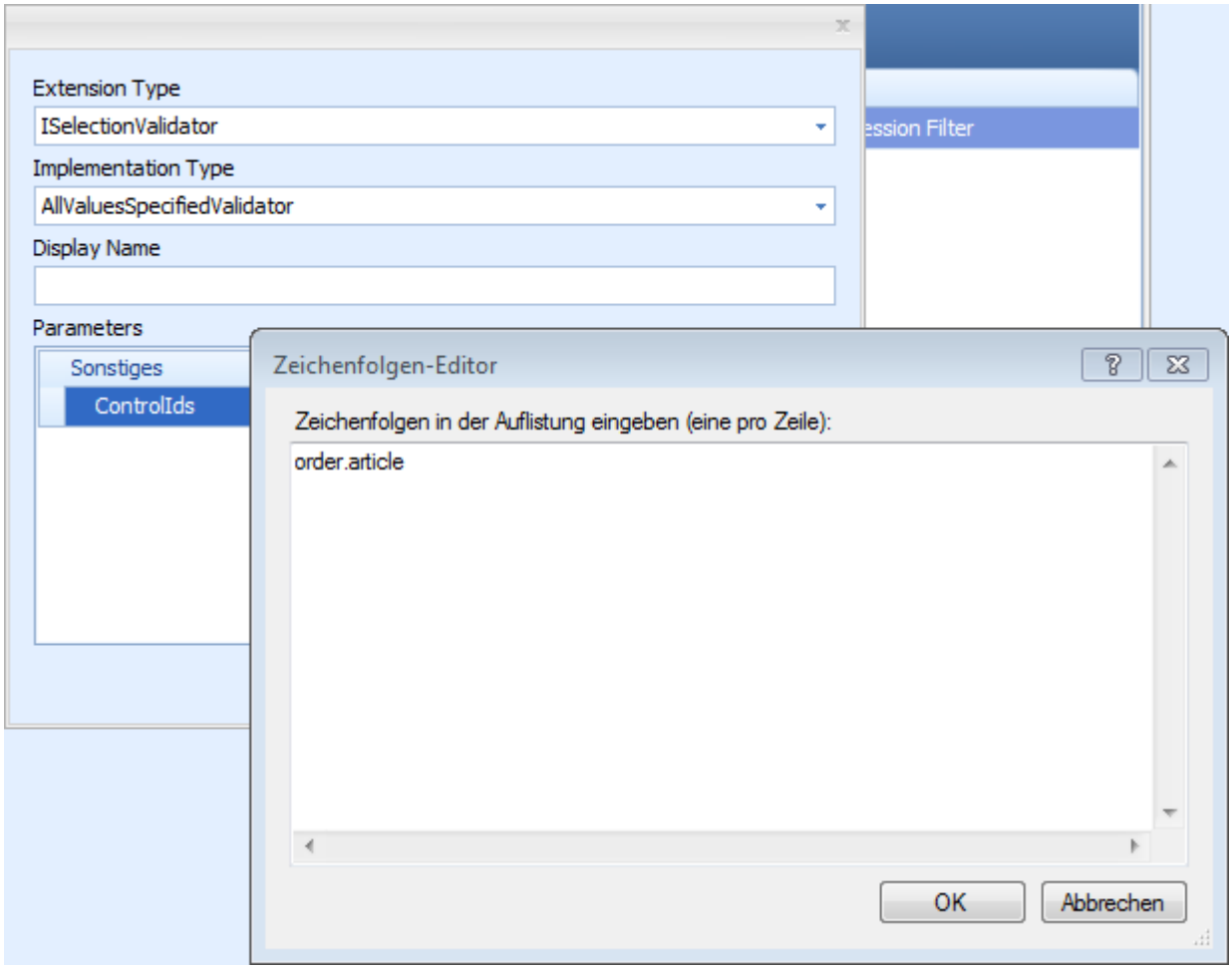
- Expression: [operation.act.yield.primary] > [operation.plan.yield.primary]
- FilterMode: AllRows (valid for all data records)
- FilterTarget: SelectedRows (for the selected data records)
- SelectedPlugin: ID of the detail application that includes the data for the expression. In this case, the table Order progress of the application Order overview. You can identify the ID of the detail application using the function (button) *Configure detail application*.

1.3.2.2 Validation before requesting data

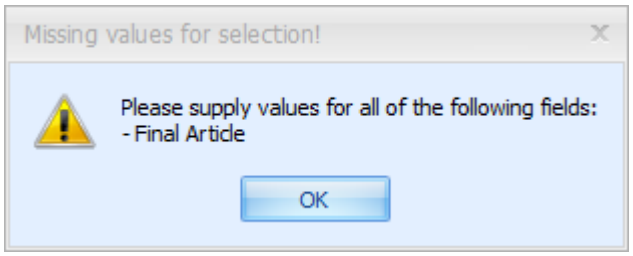
- **Category**: SelectionPanel
- **AffectedItem:** Validation
- **ItemID**: leer
- **Affected Property**: empty
- **Extension:**
  - ExtensionType: ISelectionValidator
  - Implementation Type:
    - AllValuesSpecifiedValidator
    - AtLeastOneValuesSpecifiedValidator

Example: You must enter a value in the field *Final article* in the *Order overview*.


Configure Behaviour
Configuration
OK
Cancel
Configuration
Category
SelectionPanel
Affected Item
Validation
Item ID
Affected Property
Event
Extension
AllValuesSpecifiedValidator:



If the standard extension is saved and the user does not enter a value in the selection field *Final article* before requesting data, the following error message is output:



1.3.1 Creating your own extensions

You can create your own extensions using the .NET Framework (e.g. C#, VB.NET).

1.3.1.1 Requirements

- Visual Studio Project or other .NET development environment.

- Output type: Class Library
- Output path: <MOC>\local\extensions)
- Target framework: as of 4.5.2
- Added references to Contract Libraries
    - Mpdv.Communication.Contracts.dll
    - Mpdv.Extensibility.Contracts.dll
    - Mpdv.Extensibility.Moc.Contracts.dll
    - Mpdv.ExtensionFramework.Contracts.dll
    - Mpdv.Integration.Moc.Contracts.dll
    - Mpdv.Utilities.Contracts.dll
    - Mpdv.Utilities.FileAccess.Contracts.dll
- Entry in the AssemblyInfo.cs: [assembly: ExtensionAssembly]

1.3.1.2 Creating an extension step by step

1.3.1.2.1 Requirements

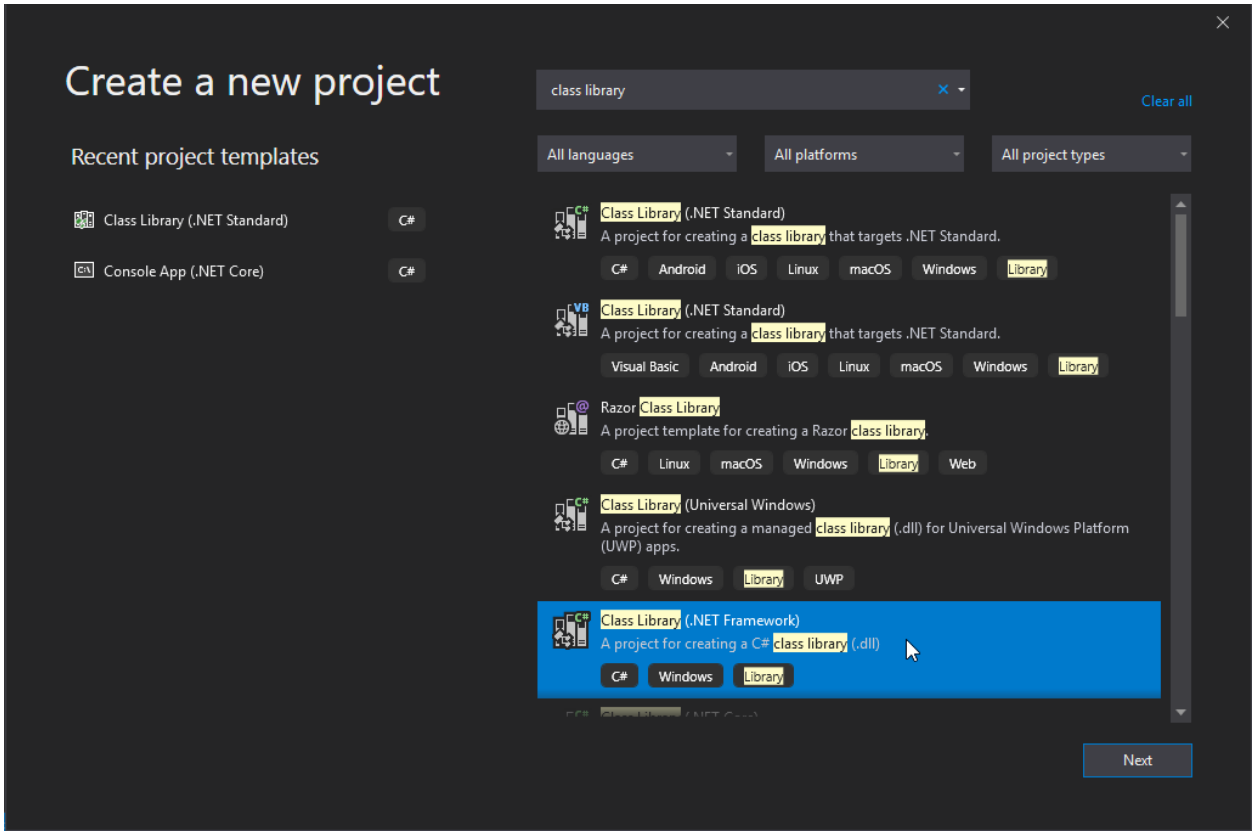
- MOC installation at the workstation (in the example in directory e:\MocNF\)
- Create the directory for the extensions in the MOC installation. In the example, this is "e:\MocNF\local\extensions\".
- Installation of "Visual Studio" at the workstation. The example was generated with an installation of Visual "Studio Community 2019".
-
    -
    -

If "Visual Studio" is not available, you can also create extensions using other .NET development environments. You can also use the free development environment "SharpDevelop". The settings that you must make are similar to the settings of this instruction.

1.3.1.2.2 Create a new project

Menu: File / New / Project

Select "Class Library (.NET Framework)"



Click [Next]

Project name: Ext_FieldUpperCase

Location: Storage location of project. In the example: "e:\DevSrc\vc\source\repos\Ext_FieldUpperCase"

Framework: .NET Framework 4.5.2

Click [Create]

1.3.1.2.3 Add references

Menu: Project / Add Reference…

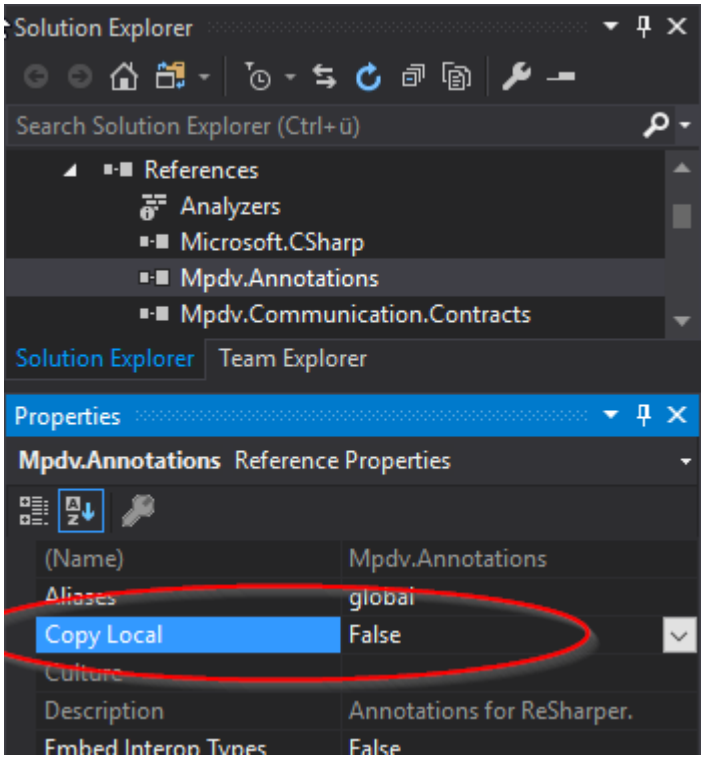
Click [Browse]

Change to the directory of the MOC installation, which contains the contracts. In the example, this is "E:\MocNF\contracts".

Add all DLLs contained in the directory. At least

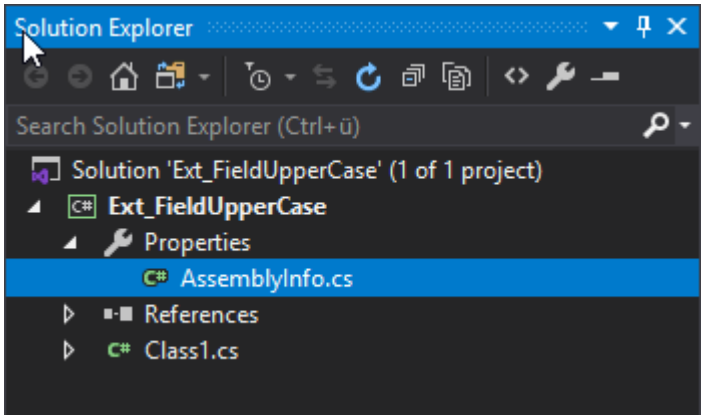
- Mpdv.Communication.Contracts.dll
- Mpdv.Extensibility.Contracts.dll
- Mpdv.Extensibility.Moc.Contracts.dll
- Mpdv.ExtensionFramework.Contracts.dll
- Mpdv.Integration.Moc.Contracts.dll
- Mpdv.Utilities.Contracts.dll
- Mpdv.Utilities.FileAccess.Contracts.dll

For all references "Mpdv.*", set the option "CopyLocal" to **False.** Otherwise, these libraries are copied into the "extensions" directory and then exist in two places.

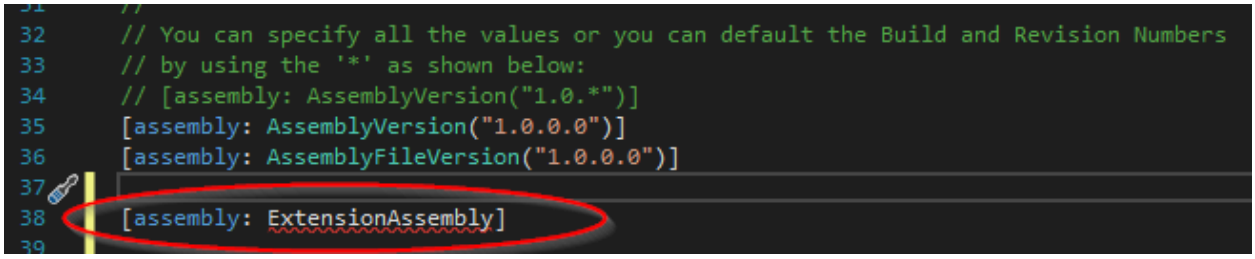


1.3.1.2.4 AssemblyInfo.cs

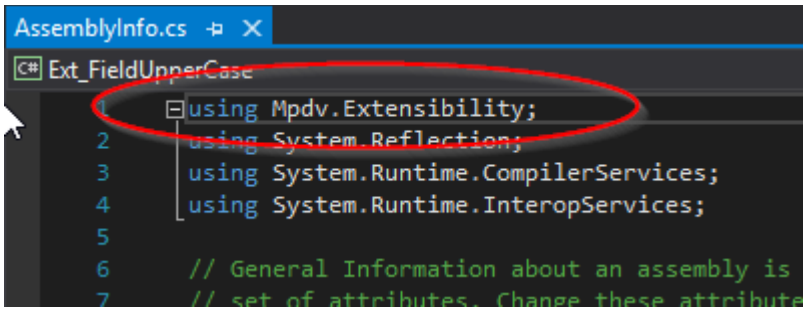
Open the file "AssemblyInfo.cs" that is included in the project.



Add a new row with the content "[assembly: ExtensionAssembly]". This row is necessary. The MOC only loads this assembly if this row is available.



At the beginning of the file, add a row with the content "using Mpdv.Extensibility" if the compiler cannot correctly interpret the previously added row.



1.3.1.2.5 Solution Properties

Menu: Project / Ext_FieldUpperCase Properties…

Select the category "Application".

Assembly name: FieldUpperCase.Extension

Default namespace: Ext.Extensions.Moc.FieldUpperCase

Target framework: Check whether the value is ".NET Framework 4.5.2".

Output type: Check whether the value is "Class Library".

Select the category "Build".

Output path: Set the output path to the sub directory "local/extensions" of your MOC installation (in the example "e:\MocNF\local\extensions\").

Save the settings (disk icon).

1.3.1.2.6 C# class

Create the C# class.

- Add the attribute [Extension]. Example: `[Mpdv.Extensibility.Extension(Name = "name of extension",Description="description..") ]`
- Derive the class from the respective interface (see developer documentation, e.g. `IControlEnterListener`).
- Implement the methods of the interface.

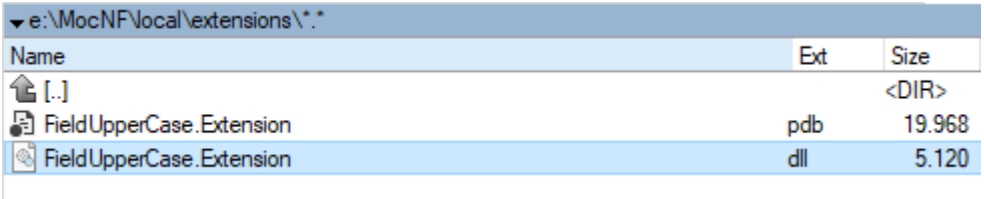
You can use one of the following examples as basis.

1.3.1.2.7 Test

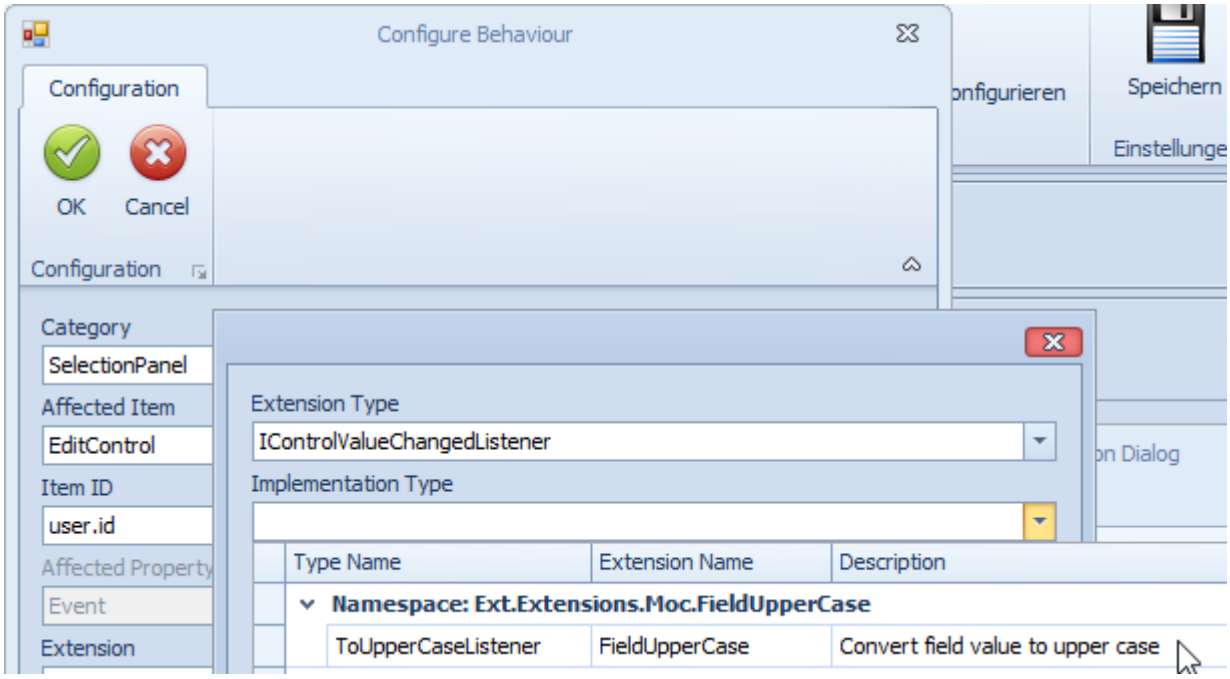
Build the solution (menu: Build / Build Solution F6)

Check and correct the errors and warnings of the compiler.

After successful Build, a DLL with the name of the extension must be available in the MOC installation (in the example in directory "e:\MocNF\local\extensions\"):



After restart of the MOC, you can enable the *MES Development Suite* and configure the new extension.



1.3.1.3 Examples

The developer documentation describes the different interfaces. Find two examples in the following:

1.3.1.3.1 The value of an input field is converted into capital letters.

When you enter a value, the value is automatically converted into capital letters. To convert the value, the system uses the interface *IControlValueChangedListener*. This interface has the method *OnEditValueChanged(..)*.

Assembly name : ValueChanged.Extension.dll

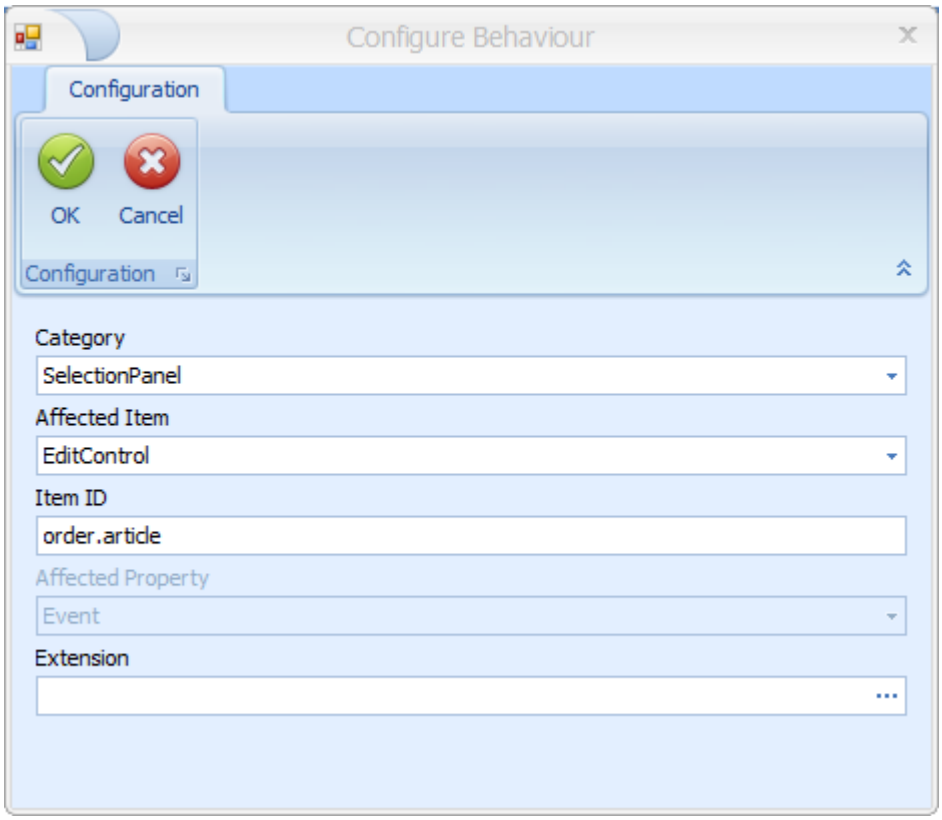
```
using System;
using System.Windows.Forms;
using Mpdv.Extensibility.Moc;
using Mpdv.Integration.Moc.Authorization;

namespace Ext.Extensions.Moc.ValueChanged
{
    [Mpdv.Extensibility.Extension(Name="FieldValueToUpper",Description="value to upper")]
    public class ToUpperCaseListener : IControlValueChangedListener
    {

        public void OnEditValueChanged(IControlContext context)
        {

            var value = context.AffectedControl.EditValue;
            if (value is String)
            {
                context.AffectedControl.EditValue = value.ToString().ToUpper();
```

```
            }
        }
    }
}
```

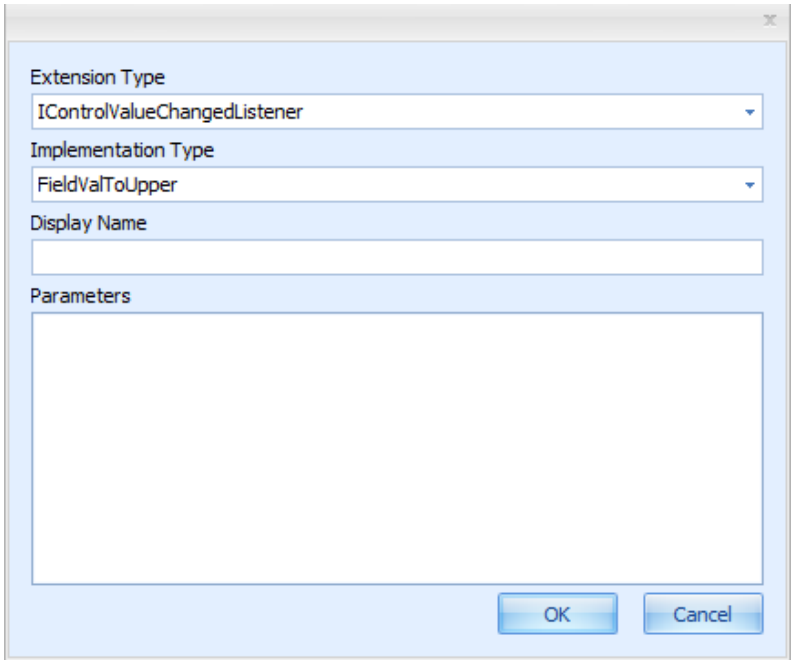
Compile into the "local\extension" directory of the MOC (ValueChanged.Extension.dll).

MOC Configuration

You want to convert the input field *Final article* (order.article) into capital letters.



The extension *FieldValToUpper* is now available as implementation type.



When you have saved the extension and restarted the application, you can test the behavior.

1.3.1.3.2 The user may only edit the own data record in the user administration

Assemblyname : MyApplicationContextFilter.extension.dll

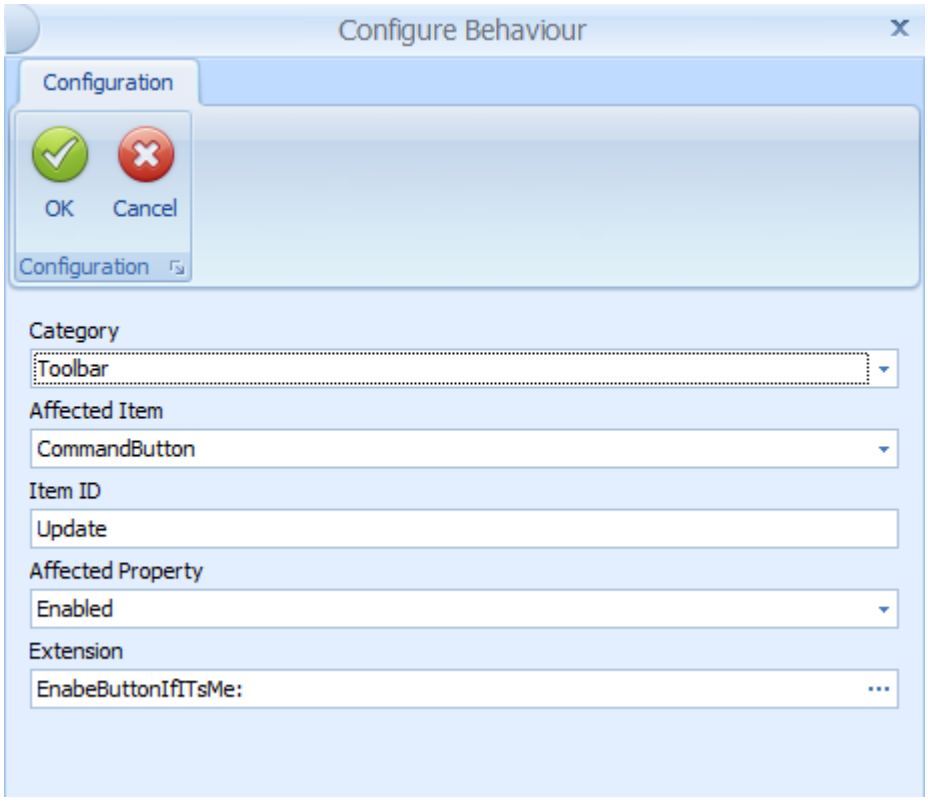
Source code:

```
namespace Extension.MyApplicationContextFilter
{
    [Extension(Description = "True, if current equals selected user")]
    [ConfigParameter("User",typeof(string))]
    public class EnabeButtonIfITsMe:IApplicationContextFilter, IConfigurable
    {
        private readonly ICurrentUserProvider currentUserProvider;
        private string userColumn;

        // constructor of class
        public EnabeButtonIfITsMe(ICurrentUserProvider currentUserProvider)
        {
            this.currentUserProvider = currentUserProvider;
        }

        // Read Parameter
        public void Configure(IConfigParameters parameters)
        {
            userColumn = parameters.GetParameter<string>("User").ToString();
        }

        // Implementation of IApplicationContextFilter method IsMatch
        public bool IsMatch(IApplicationContext context)
        {
            if (context.SelectedDataRows.Count <= 0) return false;
```

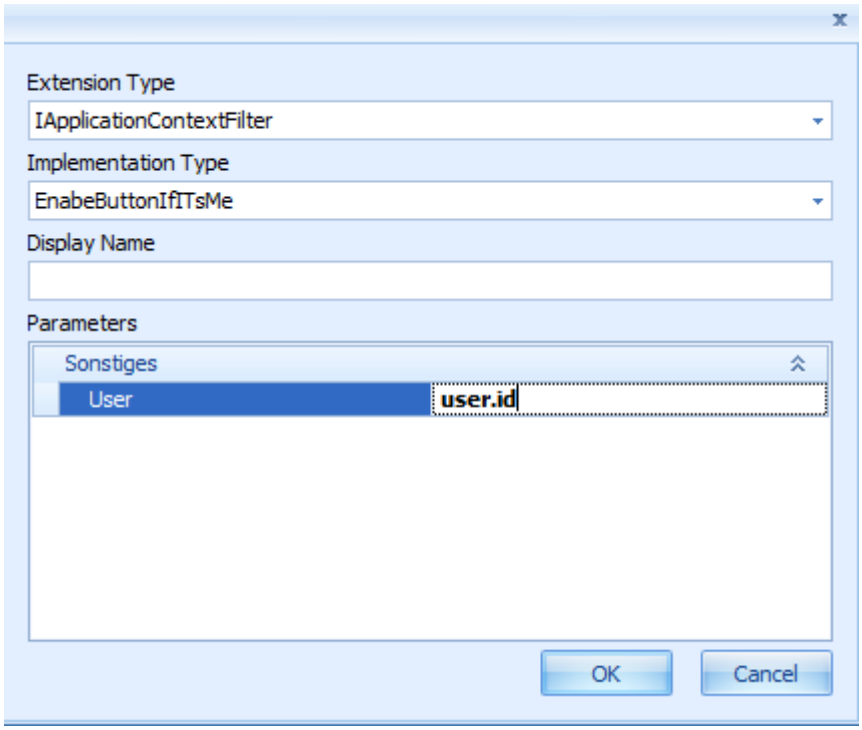
```
            var selectedUser=context.SelectedDataRows[0][userColumn];
            if (selectedUser.Equals(currentUserProvider.GetUser().UserName))
                return true;
            else
                return false;
        }
    }
}
```

Configuration of dynamic behavior of the application in the user administration on the MOC

The *Update* button may only be enabled if the user logged on to the MOC wants to edit the own data record.



The new extension "EnableButtonIfItsMe" is now available.

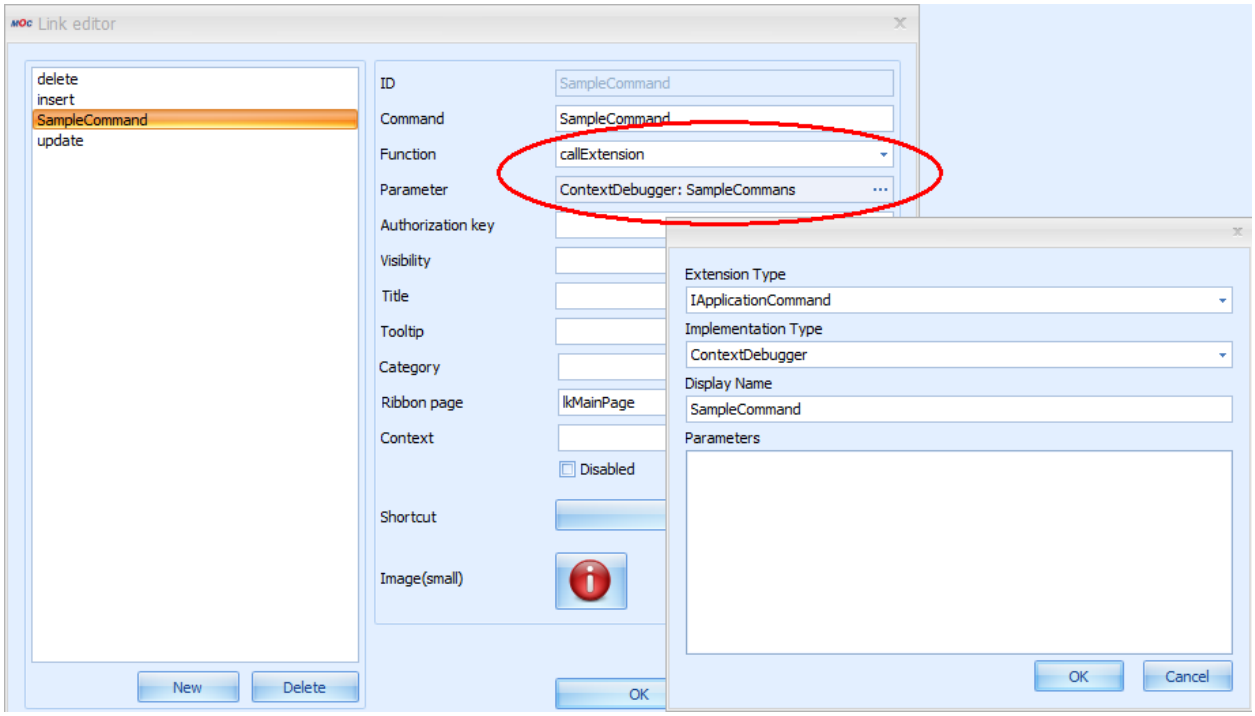


1.4 Extensions of the toolbar

If you use extensions in the toolbar, you can quit the MOC to perform external actions and then return to the MOC.

1.4.1 Configuration

You configure extensions in the toolbar using the link editor. Select the function "callExtension" and configure an extension via parameter configuration.



1.4.2 Available standard extensions

1.4.2.1 ContextDebugger

The ContextDebugger is a sample implementation that shows the current application context in a window. The application context displayed can be helpful for the development of complex applications.

1.4.3 Creating your own extensions

You can create your own extensions using the .NET Framework (e.g. C#, VB.NET).

1.4.3.1 Requirements

The same requirements apply as for the extended application configuration (section 1.3.1.1 Requirements).

1.4.3.2 Creating an extension

To extend the toolbar, you must implement the interface *IApplicationCommand.*

The interface requires the method "**Execute**" where the external processing is performed. The object *IApplicationContext* includes the information required for the request context (e.g. selected data records).

The method "Execute" returns the object `ApplicationAction`:

- The application is to be closed (`ApplicationAction.Close`).

- The application is to "Request data" (`ApplicationAction.RequestData`).
- Nothing is to happen (`ApplicationAction.None`).

You can inject additional objects in the constructor of the class that perform specific tasks (web services, localize, print/show report, logging). See also the API documentation. The API documentation is handed out as part of the training. You can also download the current version of the API documentation from the Support Portal.

1.4.3.3 Examples

The API documentation describes the different interfaces. Find two examples in the following:

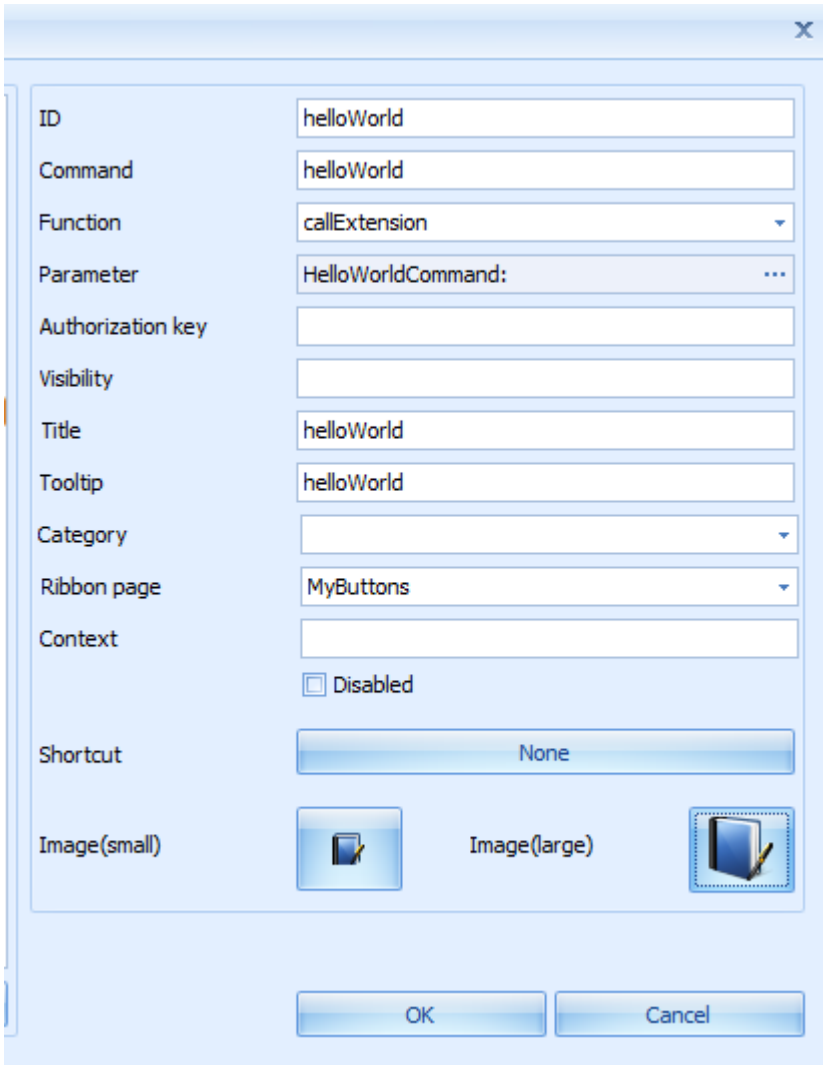
1.4.3.3.1 HelloWorld

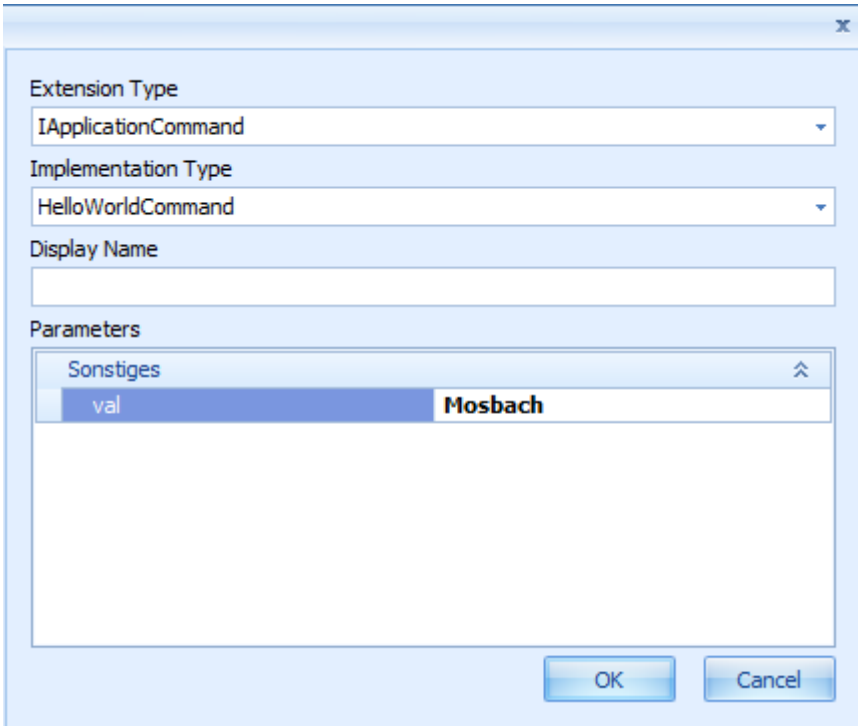
This example outputs a message box "HelloWorld". You can also add a parameter.

AssemblyName: HelloWorld.Extension

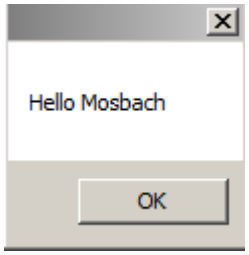
```
namespace Ext.Extensions.Moc.HelloWorld
{
    [Extension(Description = "Hello World"),ConfigParameter("val",typeof(String))]
    public class HelloWorldCommand : IApplicationCommand, IConfigurable
    {
        private string name = "";

        //constructor of class
        public HelloWorldCommand()
        {
        }

        /// <summary>
        /// Configures the instance.
        /// </summary>
        /// <param name="parameters">The parameters.</param>
        public void Configure(IConfigParameters parameters)
        {
            name = parameters.GetParameter<string>("val");
        }

        public ApplicationAction Execute(IApplicationContext applicationContext)
        {
            MessageBox.Show(String.Format("Hello {0}", name));

            return ApplicationAction.None;
        }

    }
}
```

MOC Configuration





Result



1.4.3.3.2 Example including web service call

This example shows how you can call a web service. You can call a web service using the **IRequestFactory.** You can use all web services available.

AssemblyName: WebServiceExample.Extension

```
[Extension(Description = "WebService Example")]
 public class WebServiceAndReportExample : IApplicationCommand
 {
     private readonly IRequestFactory requestFactory;

     public WebServiceAndReportExample(IRequestFactory requestFactory)
     {
         this.requestFactory = requestFactory;
     }

     /// <summary>
```

```
/// Execute Method
/// </summary>
/// <param name="applicationContext"></param>
/// <returns>ApplicationAction.RequestData</returns>
public ApplicationAction Execute(IApplicationContext applicationContext)
{

    foreach (var r in applicationContext.SelectedDataRows)
    {
       string order = r["order.id"].ToString();

        // Update Userfield 29 of order
        var config = requestFactory.GetDefaultConfiguration("BOOrder.update");
        config.Parameters.SetOrAdd("order.id", Operator.EqualTo, order);

                    config.Parameters.SetOrAdd("order.userfield29",
                    Operator.EqualTo, "PRINTED");

        IAsyncRequest request = requestFactory.CreateAsyncRequest(config);

        try
        {
            request.GetResultAsync();
        }
        catch (Exception e)
        {
            logger.Trace(e.Message);
        }
    }


    return ApplicationAction.RequestData;
}
```

1.5 Creating programmed applications

Independent of the Application Framework, you can program own applications using the .NET Framework (e.g. C#, VB.NET, etc.). That means: You must program selection panel, DataController, docking, grids, charts, etc. yourself. The basic functions of the MOC are available in self-programmed applications (web services, reports, dictionary, system information, etc.).

You can use all functions and components of the .NET Framework. You can also integrate third-party components.

To this end, the interfaces **IMocApplication (**or **IParameterizedMocApplication)** and **IMocApplicationDescriptor** are available.

1.5.1 Definition of a new application

You can completely define the application in source code. You do not require further entries in other files.

1.5.1.1 IMocApplication

You use the interface "IMOCApplication" to define the own MOC application. Using this interface, you can start your own MOC application via the transaction code or via the menu. You cannot transfer parameters to the application. Use the interface "IParameterizedMocApplication" if you want to transfer parameters.

Method	Description
RunApplication(Form mdiParent)	Main function to start own application in MOC Parameter: mdiParent: The MDI parent to attach to. If the app should not open as a child this value can be null.
Constructor of own class	Possible Parameters: IReportHelper IRequestFactory ICurrentLocalizerProvider ICurrentUserProvider IAuthorizationManager IExtensionLogger

This interface has the single method RunApplication(Form mdiParent). In this method, you can instantiate and call a self-programmed form. The constructor of the class provides the basic functions/information of the MOC.

Example:

```
[Extension]
 public class ExampleApplication : IMocApplication
 {
     IReportHelper reportHelper = null;
     IRequestFactory requestFactory = null;
     ICurrentCultureProvider cultureProvider = null;
     ICurrentLocalizerProvider localizeProvider = null;
     ICurrentUserProvider currentUSer = null;
     IAuthorizationManager authorisationManager = null;

     public ExampleApplication(  IReportHelper reportHelper,
                                 IRequestFactory requestFactory,
                                 ICurrentCultureProvider cultureProvider,
                                 ICurrentLocalizerProvider localizeProvider,
                                 ICurrentUserProvider currentUSer,
                                 IAuthorizationManager authorisationManager
                               )
     {
        this.reportHelper = reportHelper;
        this.requestFactory=requestFactory;
        this.cultureProvider = cultureProvider;
        this.localizeProvider = localizeProvider;
```

```
            this.currentUSer = currentUSer;
            this.authorisationManager = authorisationManager;
        }

        public void RunApplication(Form mdiParent)
        {
            var form = new Form1(reportHelper, requestFactory)
            {
                MdiParent = mdiParent
            };

            form.enableTimer(true);
            form.Show();

        }
    }
```

1.5.1.2 IParameterizedMocApplication

Use the interface "IParameterizedMocApplication" to define the own MOC application if you want to add parameters to the application when it is called. Using this interface, you can start your own MOC application via the transaction code or via the menu. If you do not want to pass parameters, you can optionally use the interface "IMocApplication".

The start of your own MOC applications including parameters via the interface "IParameterizedMocApplication" is available on the MOC as of service pack 15 (fall 2019).

If you implement both interfaces („IMocApplication“ and „IParameterizedMocApplication“), your MOC application is compatible with the service pack 15 status and with older MOC installations. If the own C# application implements both interfaces, the C# application is called via the method RunApplication() of the interface "ParameterizedMocApplication".

Method	Description
void RunApplication([CanBeNull] Form mdiParent, [CanBeNull] string[] args)	Main function to start own application in MOC. Parameters: mdiParent: The MDI parent to attach to. If the app should not open as a child this value can be null. Args: Parameters that are passed to the application. Can be null, in case where no parameters are passed.
Constructor of own class	Possible Parameters: IReportHelper IRequestFactory ICurrentLocalizerProvider ICurrentUserProvider IAuthorizationManager

	IExtensionLogger

This interface has the single method RunApplication(Form mdiParent, string[] args). In this method, you can instantiate and call a self-programmed form. The constructor of the class provides the basic functions/information of the MOC. The parameters of the request are passed in parameter "args".

The interface "IParameterizedMocApplication" is implemented like the interface "IMocApplication.

1.5.1.3 IMocApplicationDescriptor

The interface "IMOCApplicationDescriptor" describes the properties of the MOC application (application ID, language keys, transaction codes).

Field name/Method	Description
ApplicationId	application identifier
ApplicationNameLanguageKey	application name language key
ApplicationDescriptionLanguageKey	application description language key
AuthorizationKey	authorization key
TransactionCode	transaction code
ApplicationExtensionType	type of the application extension. Type of class of the programmed application

Note: The authorization key must have been created for the current user. Otherwise, the application is not available.

Example:

```
[Extension]
public class ExampleApplicationDescriptor : IMocApplicationDescriptor
{
    public string ApplicationDescriptionLanguageKey
    {
        get { return "lkExample"; }
```

```
        }

        Type IMocApplicationDescriptor.ApplicationExtensionType
        {
            get
            {
                return typeof(ExampleApplication);
            }
        }

        public string ApplicationId
        {
            get { return "Example"; }
        }

        string IMocApplicationDescriptor.ApplicationNameLanguageKey
        {
            get { return "lkExample"; }
        }

        string IMocApplicationDescriptor.AuthorizationKey
        {
            get { return "u_test"; }
        }

        string IMocApplicationDescriptor.TransactionCode
        {
            get { return "u_test"; }
        }
    }
```

For further details, refer to the API documentation.

1.5.1.4 Storage location

The MOC directory includes the directory "applications" in each of the scopes. You must create a directory named ApplicationId in this directory. Compile the class library in this directory.

Example: ApplicationId = Example (scope Local)

<MOC directory>\local\applications\Example\Example.MocApplication.dll

1.5.2 Integration into the MOC menu

You can integrate programmed applications into the MOC menu using the menu editor. In the menu editor, enter the transaction code of your own MOC application as command.

If you have implemented the interface "IParameterizedMocApplication" in your own MOC application, then you can enter the parameters in the column "Parameter" of the menu editor that are passed to the application via the method "RunApplication" in parameter "args".

Requirements:

- The application must implement the interfaces of the Extensibility Framework.
- The user must be authorized for the authorization key of the application (function authorization).

1.5.3 Integration into the toolbar

You can call your own MOC application created using C# via a button in the toolbar.

Make the following settings in the link editor:

Command

You can use any text for the ApplicationCommandLinks, e.g."callCommandObject".

Function

Fixed: callCommandObject

Parameters

"ApplicationId" of the implementation of the interface "IMocApplicationDescriptor". If you implement the interface "IParameterizedMocApplication", you can optionally pass further static parameters separated by blanks.

It is currently not possible to pass values from the calling application to the own MOC application called. The keys "DC" and "SP" are not dynamically resolved.

1.5.4 Calling "real MOC applications" from own applications

You can call "real MOC applications" from own MOC applications on the MOC as of service pack 15 (fall 2019).

The class "**MocCommandHelper"** is integrated in the Extension Framework, which implements the interface "**ICommandHelper"** and provides the relevant methods for the request of MOC applications. You can inject "MocCommandHelper" in a C# application if you specify the "ICommandHelper" interface in the constructor.

You can use the method "**OpenApplication**()" to call MOC applications that have the type MOC application. These are all applications that can be listed on the MOC using the transaction code "_cselectApp". You can also call own C# MOC applications. When you call the relevant variant, you can also pass static parameters to the MOC application or the C# application.

Use the method "**RetrieveApplicationData**()" to call an MOC application as pool application. In the call, you can optionally pass static parameters separated by blanks to the pool application. The data selected in "pool mode" is returned as result of the function call to the calling application.

```
using System.Collections.Generic;
namespace Mpdv.Integration.Moc.Execution
{
    /// <summary>
    /// Class for accessing functions of the command framework.
    /// </summary>
    public interface ICommandHelper
    {
        /// <summary>
        /// Opens the MOC application to the specified identifier.
        /// </summary>
        /// <param name="id">The identifier.</param>
        void OpenApplication(string id);

        /// <summary>
        /// Opens the MOC application to the specified identifier.
        /// </summary>
        /// <param name="id">The identifier of the application.</param>
        /// <param name="parameters">Parameters that are passed to the application (multiple parameters
        /// must be delimited with blank character).</param>
        void OpenApplication(string id, string parameters);

        /// <summary>
        /// Opens the MOC application to the specified identifier as pool application and returns the selected
        /// values of the application.
        /// </summary>
        /// <param name="id">The identifier of the application.</param>
        /// <param name="parameters">Parameters that are passed to the application (multiple parameters must be delimited with
        /// blank character).</param>
        /// <returns>The selected values as key/value pair where the key contains the name and the value contains
        /// the value of the field.</returns>
        IEnumerable<KeyValuePair<string, object>> RetrieveApplicationData(string id, string parameters);
    }
}
```

Example:

```
using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;
using Mpdv.Integration.Moc.Execution;

namespace MyParameterizedSampleMocApplication
{
    public partial class MainForm : Form
    {
        private readonly ICommandHelper _commandHelper;
        private string _arguments;

        public string Arguments
        {
            get => _arguments;
            set
            {
                _arguments = value;
                argumentsRichTextBox.Text = value;
            }
        }

        public MainForm(ICommandHelper commandHelper)
        {
            _commandHelper = commandHelper;
            InitializeComponent();
        }

        private void OpenApplicationButton_Click(object sender, System.EventArgs e)
        {
            if (string.IsNullOrEmpty(applicationParameterTextBox.Text))
            {
                _commandHelper.OpenApplication(applicationIdTextBox.Text);
        return;
            }

            _commandHelper.OpenApplication(applicationIdTextBox.Text, applicationParameterTextBox.Text);
        }

        private void RetrieveDataViaApplicationButton_Click(object sender, System.EventArgs e)
        {
            var retrieveDataViaApplication = _commandHelper.RetrieveApplicationData(applicationIdTextBox.Text,
applicationParameterTextBox.Text);
            dataGridView1.DataSource = retrieveDataViaApplication?.
                Select(pair => new KeyValuePair<string, string>(pair.Key, pair.Value?.ToString())).ToList();
        }
    }
}
```